Project - High Level Design

On

Multi-Agent Manufacturing System

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	AADITI BHALE	EN22CS303001
2.	ADITYA PUROHIT	EN22CS303007
3.	ABHIGYAN SINGH THAKUR	EN22CS303004
4.	DHRUV JAIN	EN22CS304022
5.	DIVYANSH TYAGI	EN22CS304024

Group Name: 06D11

Project Number:032

Industry Mentor Name: AASHRUTI SHAH

University Mentor Name: NISHANT SHRIVASTAV

Academic Year:2022-2026

Table of Contents

1. **Introduction.**
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. **System Design.**
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. **Data Design.**
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. **Interfaces**

5. **State and Session Management**

6. **Caching**

7. **Non-Functional Requirements**
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1.Introduction:

This document describes the **High-Level Design (HLD)** of the **Multi-Agent Manufacturing System**, an Agentic AI-based application.

The system leverages multiple intelligent agents that collaborate to perform tasks such as:

- Manufacturing research
- Supplier sourcing
- Content generation
- Visualization

The architecture focuses on:

- Agent collaboration
- Task orchestration
- Context sharing
- Scalable deployment

1.1 Scope of the Document

This document covers:

- High-level architecture of the system
- Agent-based workflow design
- System components and interactions
- API structure
- Deployment overview
- Non-functional requirements

1.2 Intended Audience

This document is intended for:

- AI/ML Engineers
- Backend Developers
- DevOps Engineers
- Academic Evaluators
- Project Review Committee

1.3 System Overview

The system is an **Agentic AI architecture** where multiple specialized agents collaborate to generate structured manufacturing insights.

Key capabilities:

- Multi-agent collaboration
- Automated report generation
- Context-aware processing
- Scalable microservice deployment

The system consists of:

- Frontend UI
- Flask backend
- Agent Orchestrator (CrewAI/LangChain)
- Specialized agents (Research, Writer, Design)
- LLM APIs (OpenRouter)
- Docker + Kubernetes deployment

2. System Design

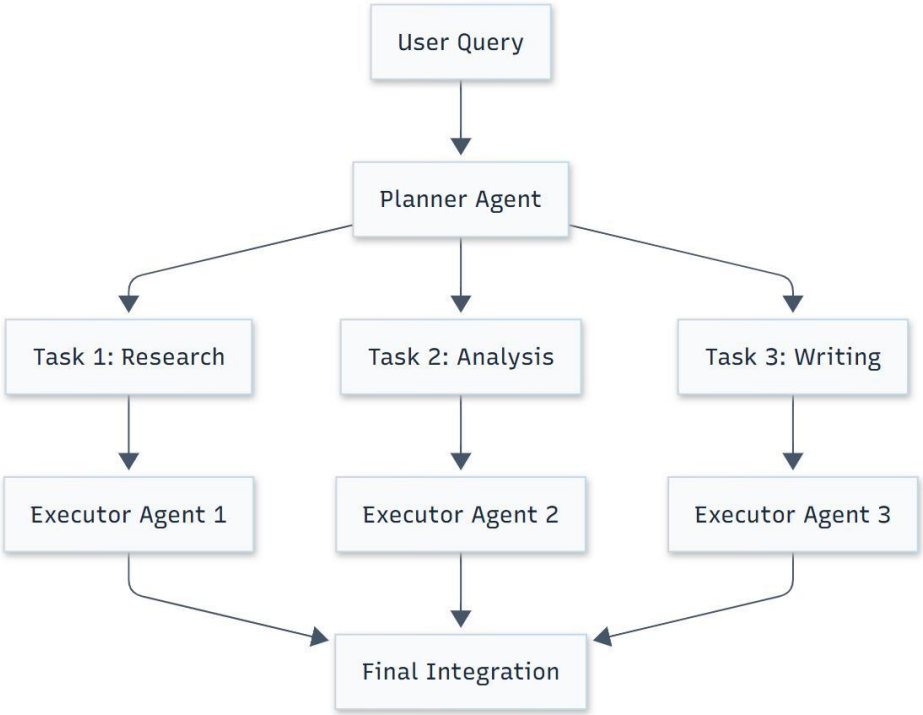
2.1 Application Design

The application follows a **layered agent-based architecture**:

Core Functional Modules:

- Prompt Processing
- Agent Execution
- Response Aggregation
- Output Rendering

Flow:



2.2 Process Flow - Deployment Flow

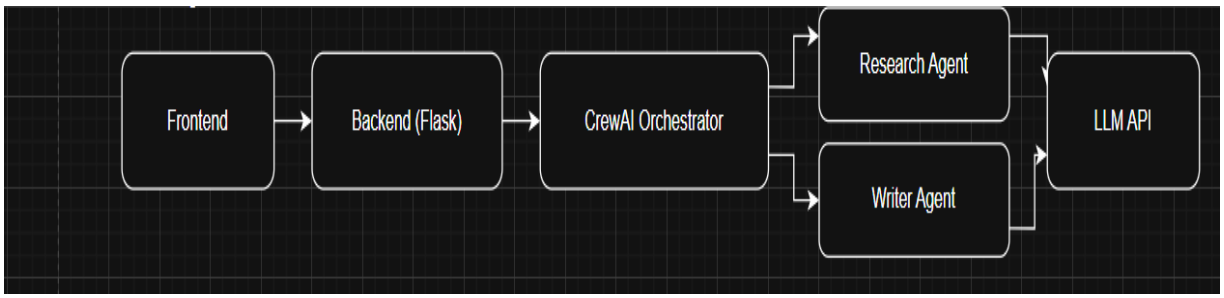
- User enters manufacturing prompt
- Frontend sends request to backend
- Backend initializes agent orchestration
- Planner divides task
- Research Agent gathers data
- Writer Agent formats structured output

- (Optional) Design Agent enhances output
- Backend aggregates responses
- Final output sent to user

2.3 Information Flow

- Input: User manufacturing query
- Intermediate: Context passed between agents
- Output: Structured manufacturing report + optional image

Flow:



2.4 Component Design

Component 1: Frontend

- Captures user input
- Displays structured output

Component 2: Flask Backend

- Handles API requests
- Manages workflow
- Aggregates responses

Component 3: Agent Orchestrator

- Controls execution of agents
- Manages task flow and context

Component 4: Research Agent

- Performs supplier sourcing
- Gathers manufacturing data

Component 5: Writer Agent

- Converts raw data into structured format

Component 6: LLM APIs

- OpenRouter

Component 7: Docker

- Containerizes application

Component 8: Kubernetes

- Manages deployment and scaling

2.5 Key Design Considerations

- Modular agent-based design
- Scalability using Kubernetes
- Separation of responsibilities (agents)
- Stateless backend architecture
- Extensibility for adding new agents
- Fault tolerance via container orchestration

2.6 API Catalogue

1. Home API

- **Endpoint:** /
- **Method:** GET
- **Description:**
  Provides the frontend interface for user interaction. Displays input field and triggers agent workflow.
- **Request:**
  None

2. Generate API (Core Agentic Endpoint)

- **Endpoint:** /generate
- **Method:** POST
- **Description:**
  This is the **core API of the system**. It triggers the **multi-agent workflow**:
  - Initializes agents
  - Executes Research Agent
  - Passes context to Writer Agent
  - Returns structured output
- **Request Body:**

```
{
  "prompt": "Smart manufacturing system for electric vehicles"
}
```

- **Response:**

```
{
  "output": "Structured manufacturing report..."
}
```

3. Health Check API

- **Endpoint:** /health
- **Method:** GET
- **Description:**
  Used by Kubernetes for:
  - Liveness probe
  - Readiness probe
- **Response:**

"OK"

3. Data Design

3.1 Data Model

Currently:

- No database (stateless system)

Future:

- Vector database for RAG
- Structured manufacturing dataset

3.2 Data Access Mechanism

- API-based communication
- LLM-based data generation
- No persistent storage

3.3 Data Retention Policies

- No data stored permanently
- Data exists only during request lifecycle

3.4 Data Migration

Not applicable (stateless system)

4. Interfaces

External Interfaces

- User interacting via UI

Internal Interfaces

- Backend ↔ Agent Orchestrator
- Agent ↔ Agent communication

AI Interfaces

- OpenRouter API

Deployment Interfaces

- Docker Hub
- Kubernetes cluster

4.1 External API Interface

4.1.1 Home Endpoint Interface

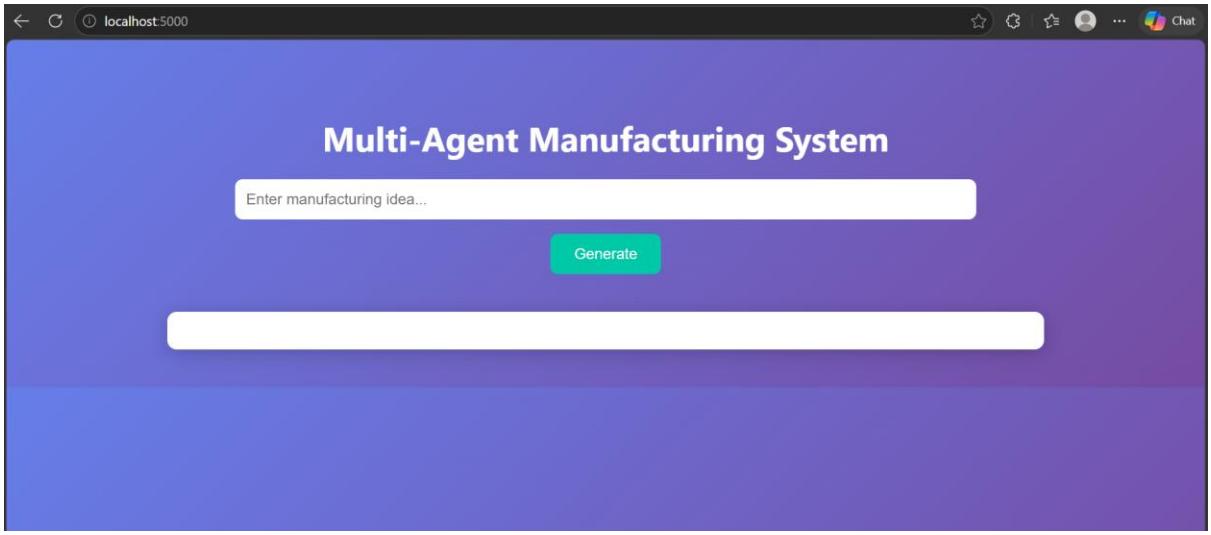


Figure 4.1: Home API Endpoint Output (Version Verification Interface)

The root endpoint (/) provides a user interface where users input manufacturing queries. It acts as the primary interaction point between the user and the multi-agent system.

4.1.2 Generate API Interface

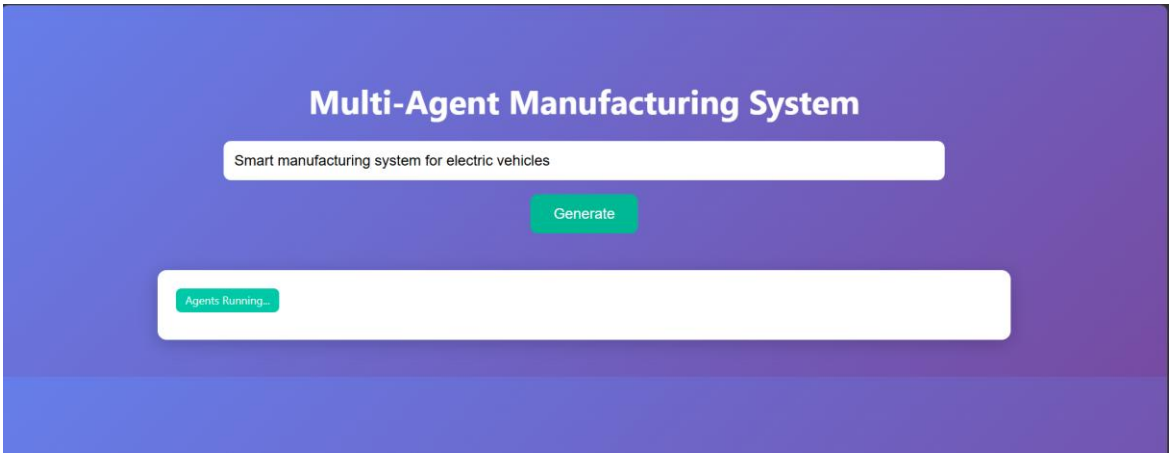
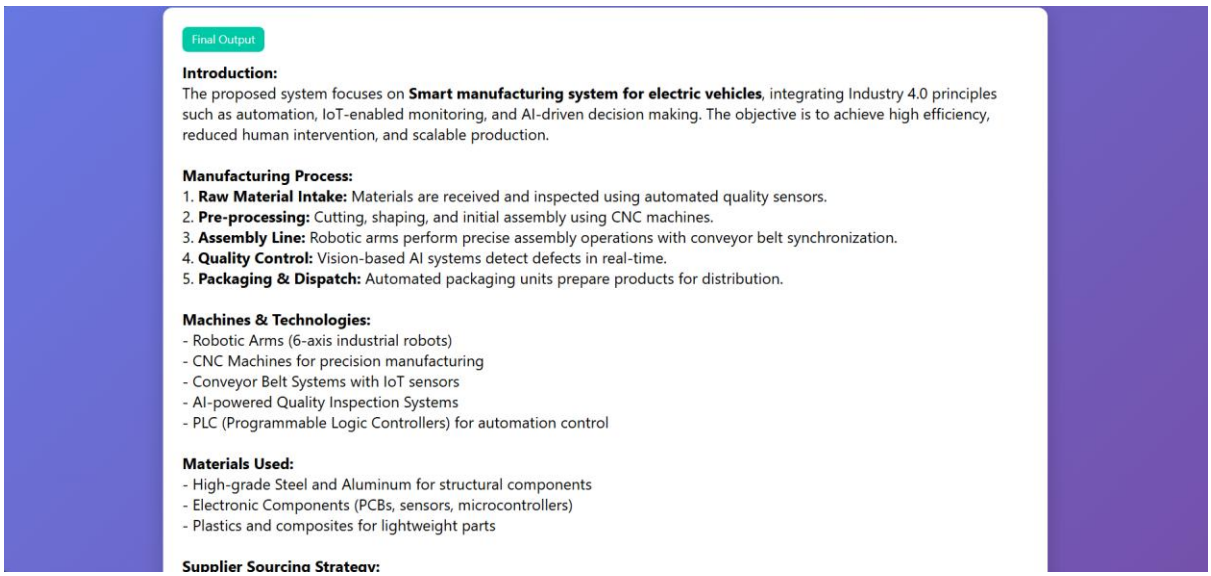
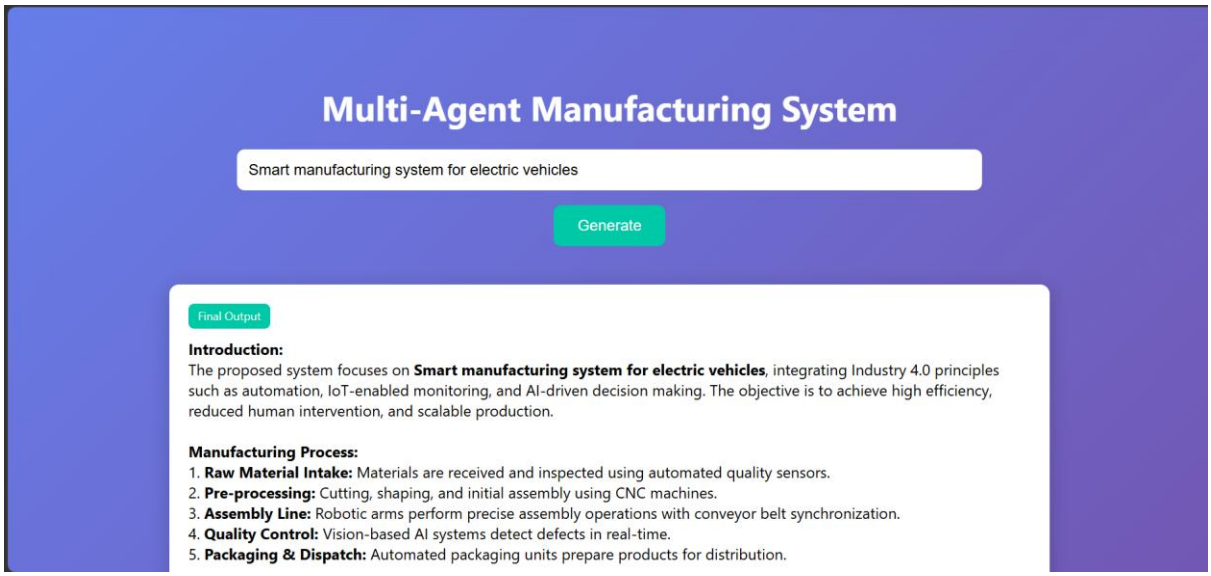


Figure 4.2: Generate API JSON Response Interface

The /generate endpoint is the core API of the system. It triggers the multi-agent workflow where the Research Agent gathers data and the Writer Agent formats it into a structured report.





4.1.3 Health Check Interface

Figure 4.3: Health Check Interface Used by Kubernetes Probes

The /health endpoint acts as an internal monitoring interface. Kubernetes uses this endpoint for liveness and readiness probes to ensure that traffic is routed only to healthy pods.

4.2 Container Registry Interface

4.2.1 Docker Hub Repository Interface

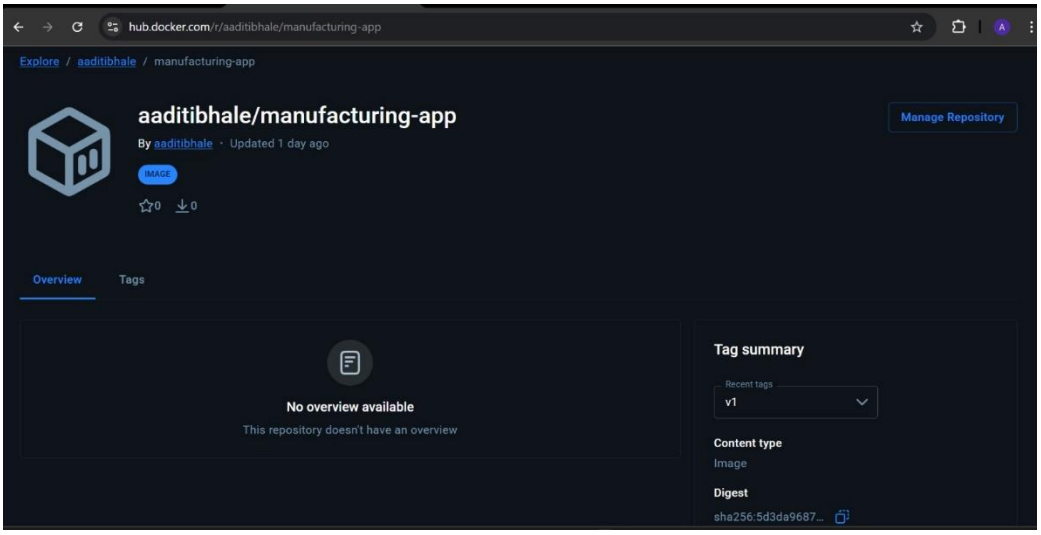


Figure 4.4: Docker Hub Repository Interface Showing Image Tag v1

Docker Hub acts as the container registry interface between the development environment and Kubernetes cluster. The application image is pushed to Docker Hub, where it is versioned and stored.

Kubernetes pulls the container image from Docker Hub during deployment.

This interface ensures portability and centralized image management.

4.4 Kubernetes Deployment Interface

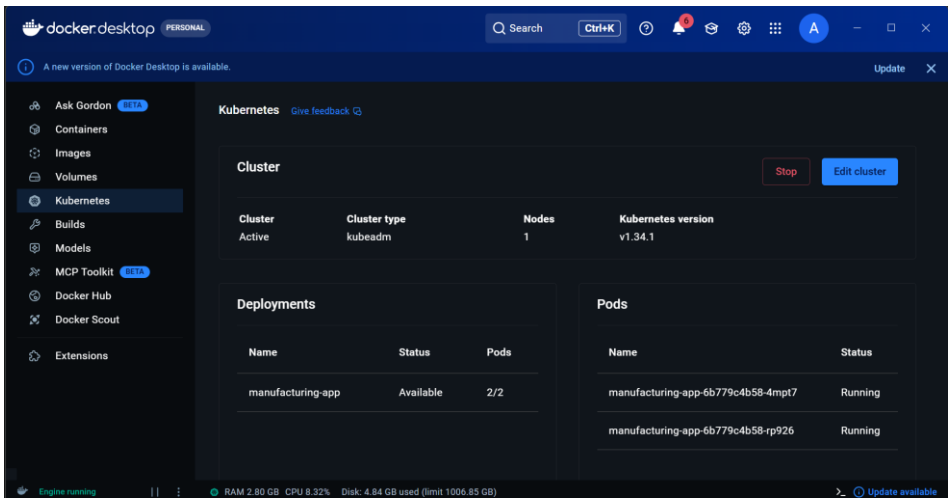


Figure 4.6: Container Registry Interface Used by Kubernetes Deployment

Kubernetes interacts with Docker Hub through image pull operations. When the deployment.yaml file is applied using kubectl, Kubernetes automatically pulls the specified image version from Docker Hub.

5.State and Session Management

5.1 Overview

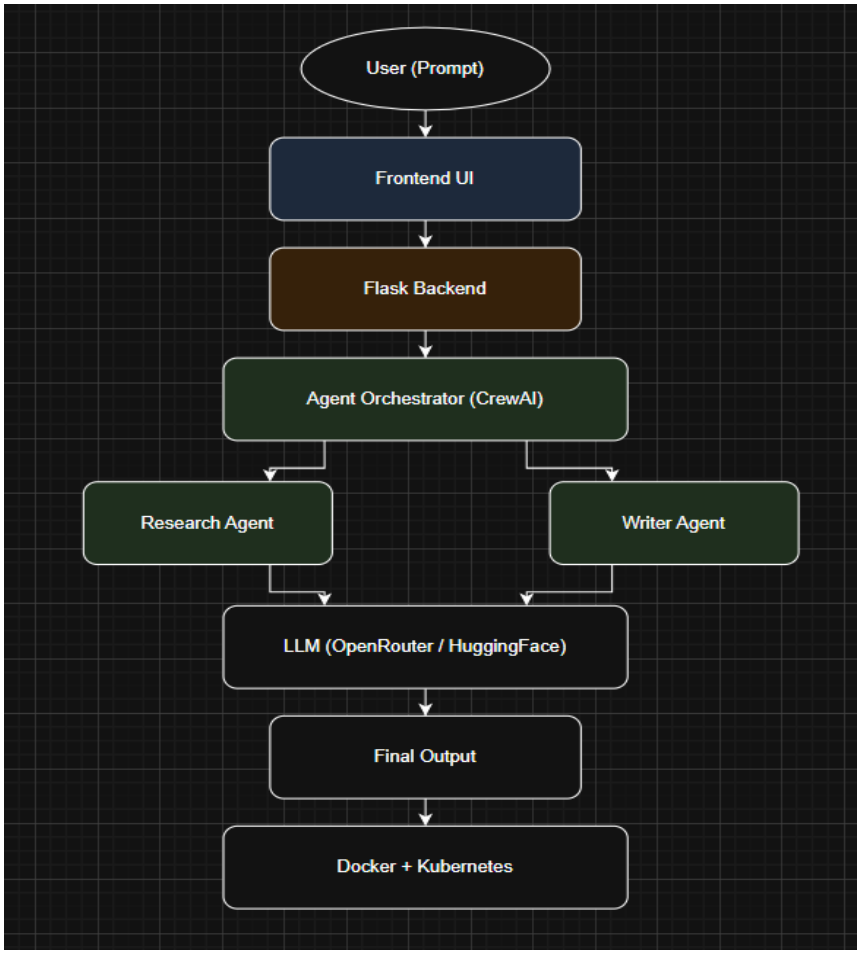
The system is **stateless**:

- No session storage
- Each request processed independently

Future enhancement:

- JWT authentication
- Redis session storage

5.4 Architecture Design



Layered Architecture Explanation:

1. User Layer

- Interacts with system
- Provides input

2. Presentation Layer (Frontend)

- Displays UI
- Sends requests

3. Application Layer (Backend)

- Handles logic
- Manages agent workflow

4. Agent Orchestration Layer

- Controls agent execution
- Manages task flow

5. AI Layer

- LLM APIs (OpenRouter)

6. Deployment Layer

- Docker containers
- Kubernetes cluster

6. Caching

Current:

- No caching implemented

Future:

- Redis caching
- API response caching
- Reduced LLM calls

7. Non-Functional Requirements

7.1 Security Aspects

- API keys stored in environment variables
- No hardcoded credentials
- Container isolation (Docker)
- Kubernetes network abstraction

Future:

- HTTPS
- RBAC
- Token-based authentication

7.2 Performance Aspects

Scalability

- Kubernetes replicas
- Stateless architecture

Availability

- Rolling updates
- Health checks

Reliability

- Self-healing pods
- Fault tolerance

Efficiency

- Lightweight Flask backend
- Optimized API calls

8. References

[1] Kubernetes, "Kubernetes Documentation," Kubernetes Official Website.
Available: https://kubernetes.io/docs/
Accessed: Apr. 2026

[2] Docker Inc., "Docker Documentation," Docker Official Website.
Available: https://docs.docker.com/
Accessed: Apr. 2026

[3] Pallets Projects, "Flask Documentation," Flask Official Documentation.
Available: https://flask.palletsprojects.com/
Accessed: Apr. 2026

[4] OpenRouter, "OpenRouter API Documentation," OpenRouter.
Available: https://openrouter.ai/docs
Accessed: Apr. 2026

[5] Meta AI, "LLaMA 3: Open Foundation and Instruction-Tuned Models," Meta AI.
Available: https://ai.meta.com/llama/
Accessed: Apr. 2026

[6] CrewAI, “CrewAI Documentation,” CrewAI Official Docs.
Available: https://docs.crewai.com/
Accessed: Apr. 2026

[7] LangChain, “LangChain Documentation,” LangChain Official Docs.
Available: https://python.langchain.com/docs/
Accessed: Apr. 2026

[8] C. Richardson, *Microservices Patterns*, Manning Publications, 2018.

[9] J. Humble and D. Farley, *Continuous Delivery*, Addison-Wesley, 2010.

[10] The Linux Foundation, “Cloud Native Computing Foundation (CNCF),”
Available: https://www.cncf.io/
Accessed: Apr. 2026